
ML-Driven Inverse Design for Refrigerant Separation

A milestone-based bootcamp that converts simulation expertise (GCMC, MLIPs, free energy, force-field fitting) into the generative-ML, uncertainty, and autonomous-discovery skill set targeted by computational materials roles.

Anchor problem

Design MOF / MOSC linkers that selectively separate **R-32 from R-125** (the R-410A azeotrope), with your own simulations acting as the validating “automated lab.”

Format	Self-paced course, 7 modules, ~25 lessons
Pace	~7–9 months part-time (alongside thesis)
Method	Toy-from-scratch → apply to project → closed-book checkpoint
Output	2 papers + a portfolio repo mapped to job descriptions

How to use this course (read first)

The discipline below *is* the curriculum. Skip it and you will end up with a repository but not the skill — and the gap surfaces in exactly the technical interviews where you can't bring an AI.

- **Toy first, from a blank file.** Every concept has a ~50-line toy that isolates it. Build that by hand before touching real data or any library that hides the idea. The toy is where the math turns into intuition.
- **Checkpoints are gates.** Each lesson ends with a closed-book checkpoint. Failing it is information about what to restudy, not a reason to push on.
- **Primary source before summary.** Read the listed papers first; use AI to clarify only *after* you have wrestled with the real text.
- **This handbook withholds full solutions on purpose.** You get the theory, the build steps, skeletons, and gotchas. You write the working code. Bring it to a mentor or to AI for *review*, not authorship.
- **Write one paragraph of “what I learned” per lesson into the repo.** Explaining it back is what makes it stick.

How each lesson is structured

Objectives (what you can do after) → **The idea** (math + science, explained) → **Build it** (numbered steps, toy then project) → **Skeleton** (signatures/pseudocode, never the full answer) → **Deliverable** → **Checkpoint** → **Read**.

Course map

Module 0 — Foundations & setup

- 0.1 Reproducible project skeleton
- 0.2 The reusable math, learned by coding it
- 0.3 A training loop from a blank file

Module 1 — The science & the dataset

- 1.1 Why R-32/R-125 is hard + IAST selectivity
- 1.2 GCMC ground truth & its statistical mechanics
- 1.3 Refrigerant force fields vs. your MLIP
- 1.4 Featurize & curate a leakage-aware dataset

Module 2 — Surrogate model + uncertainty

- 2.1 Dimensionality reduction the honest way
- 2.2 Baseline regression & screening metrics
- 2.3 A graph neural network from message passing up
- 2.4 Uncertainty quantification & calibration

Module 3 — Generative models (headline skill)

- 3.1 Representing a linker
- 3.2 The VAE from the ELBO up
- 3.3 Diffusion from scratch
- 3.4 Property-conditioned generation

Module 4 — Physics-informed steering / RL

- 4.1 RL fundamentals through code
- 4.2 Reward design with your MLIP
- 4.3 Steering the generator

Module 5 — Closed-loop validation & evaluation

- 5.1 Evaluation metrics from scratch
- 5.2 Active learning loop
- 5.3 Wire the full loop

Module 6 — Orchestration, agents & portfolio

- 6.1 Reproducible pipeline
- 6.2 Optional agentic layer
- 6.3 Portfolio & job-description mapping

MODULE

0 · Foundations & setup

1–2 weeks

Before any materials science, you lock in reproducibility and meet the four pieces of math you will reuse in every later module — KL divergence, the log-derivative trick, autodiff, and a hand-written training loop. Meeting them now in tiny isolation is what makes the VAE, diffusion, UQ and RL modules feel like variations on things you already know.

Reuses what you already have:

Python/Bash fluency, HPC habits, your existing scientific-computing workflow.

0.1 Reproducible project skeleton

~3 days

WHAT YOU'LL WALK AWAY ABLE TO DO

- Stand up a project that rebuilds bit-for-bit from a lockfile
- Separate config from code so experiments are declarative

THE IDEA

Reproducibility is the first JD bullet (“modular, reproducible pipelines”) and the cheapest credibility you can buy. The goal is that a stranger — or you in six months — can recreate any result from a commit hash plus a lockfile.

Config-driven means every run is described by a YAML file, not by edited source. That single habit makes sweeps, active learning, and the agentic layer in Module 6 trivial later.

BUILD IT

- Create an environment with a pinned lockfile (uv or conda-lock). Commit the lock.
- Lay out folders: data/ src/ configs/ notebooks/ tests/ docs/.
- Write a 10-line config loader that reads a YAML and returns a typed object.
- Add a Makefile or task runner with setup / test / run targets.

Project contract you are implementing

```
configs/exp.yaml      # all knobs live here
src/data.py           # load + split, no hard-coded paths
src/model.py          # model defn
src/train.py          # reads config, logs metrics, saves artifacts
make run CONFIG=configs/exp.yaml
```

DELIVERABLE

Done when: you delete the env, rebuild from the lockfile, and a dummy run reproduces.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ Wipe the environment and reproduce a dummy run from the lockfile alone. If anything is missing, fix it now — every later module compounds on this.

0.2 The reusable math, learned by coding it

~1 week

WHAT YOU'LL WALK AWAY ABLE TO DO

- Compute a KL divergence two independent ways and trust the result
- State and demonstrate the log-derivative (score-function) identity
- Differentiate anything with autodiff and sanity-check the gradient

THE IDEA

KL divergence $KL(q \parallel p)$ measures how far distribution q is from p . For two Gaussians it has a closed form; it also equals an expectation you can estimate by sampling. You will see this exact term as the regularizer in the VAE loss (3.2).

Log-derivative trick: the gradient of an expectation over a parameterized distribution can be written as an expectation of (gradient of log-prob) times the quantity inside. That single identity *is* REINFORCE (4.1) — learning it now means RL later is not a new idea, just an application.

Autodiff is the engine under every model here: you write the forward computation, the framework gives you exact gradients. The only skill is trusting and checking them.

BUILD IT

- Derive $KL(N(m1,s1) \parallel N(m2,s2))$ on paper, then implement both the closed form and a Monte-Carlo estimate; confirm they converge as samples grow.
- Pick a scalar $f(\theta) = E[g(x)]$ with $x \sim p_\theta$; estimate its gradient with the log-derivative trick and verify against a finite-difference gradient.
- Implement gradient descent on a 2-D bowl by hand, then redo it with autodiff; confirm the step vectors match to numerical precision.

Skeleton — fill the TODOs yourself

```
def kl_gaussians(m1,s1,m2,s2):      # closed form
    return ... # TODO derive on paper first

def kl_mc(m1,s1,m2,s2,n=100_000):  # sampling estimate
    x = m1 + s1*randn(n)
    return mean(logq(x) - logp(x)) # TODO logq, logp

assert abs(kl_gaussians(...) - kl_mc(...)) < tol
```

DELIVERABLE

Done when: a notebook where closed-form and sampled KL agree, and your hand gradient matches autodiff and finite differences.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ From a blank file, write the closed-form Gaussian KL and explain in two sentences why the sampled version converges to it.
- ✓ State the log-derivative identity and explain where it will reappear in RL.

Read: any rigorous autograd primer; the reparameterization-trick section you will revisit in 3.2.

0.3 A training loop from a blank file

~3 days

WHAT YOU'LL WALK AWAY ABLE TO DO

- Write a complete PyTorch training loop with no references
- Diagnose under/overfitting from the loss curves you produce

THE IDEA

Every model in this course — GNN, VAE, diffusion, policy — is trained by the same loop: forward, loss, backward, step, evaluate, stop. If this loop is automatic for you, the later modules are about *what* to compute, never *how* to train.

BUILD IT

- Build a tiny MLP regressor on synthetic data.
- Hand-write the loop: batching, forward, loss, backward(), optimizer step, zeroing grads, a held-out eval, and early stopping.
- Plot train vs. val loss; deliberately overfit a small set, then regularize it back.

The shape you must be able to reproduce blank-file

```
for epoch in range(E):
    for xb, yb in loader:
        opt.zero_grad()
        loss = mse(model(xb), yb)
        loss.backward(); opt.step()
    val = evaluate(model, val_loader)    # early-stop on this
```

DELIVERABLE

Done when: a reproducible loop with train/val curves and early stopping.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ Reproduce the entire loop from a blank file, no references. This is non-negotiable; the rest of the course assumes it is reflexive.

MODULE

1 · The science & the dataset

3–4 weeks

You turn the separation problem into a labeled learning problem. By the end you understand why R-32/R-125 resists distillation, how adsorptive selectivity is computed with IAST, what GCMC is actually doing, and how to build a descriptor dataset whose train/test split is honest. This module is mostly your existing simulation skills, repointed and made rigorous.

Reuses what you already have:

RASPA/GCMC, MLIP force-field validation (your CF_4/CH_4 methodology), CIF handling, HPC.

1.1 Why R-32/R-125 is hard + IAST selectivity

~1 week

WHAT YOU'LL WALK AWAY ABLE TO DO

- Explain in thermodynamic terms why distillation fails for an azeotrope
- Compute mixture selectivity from single-component isotherms via IAST

THE IDEA

R-410A is a near-azeotropic 50/50 mass blend of R-32 and R-125: the vapor and liquid have nearly the same composition, so boiling does not enrich either component — distillation stalls. Adsorptive separation sidesteps this by exploiting that a porous solid can prefer one molecule.

An **isotherm** is loading vs. pressure for one gas. **IAST** (Ideal Adsorbed Solution Theory) predicts how a *mixture* partitions using only the pure-component isotherms — the thermodynamic analogue of Raoult's law for adsorbed phases. **Selectivity** is the ratio of adsorbed mole fractions normalized by gas-phase fractions; it is the number your generative model will ultimately optimize.

BUILD IT

- Generate synthetic single-component data and fit a **Langmuir** isotherm yourself with nonlinear least squares before using any adsorption library — this makes the saturation/affinity parameters concrete.
- Derive the IAST equal-spreading-pressure condition on paper.
- Use pyIAST on R-32 and R-125 isotherms to get selectivity at ~298 K; cross-check the dilute limit against your hand-fit Langmuir constants.

Langmuir to fit by hand first

```
# q(P) = q_sat * b*P / (1 + b*P)
popt,_ = curve_fit(langmuir, P, q, p0=[qsat0,b0])
# only AFTER this, call pyIAST for the mixture
```

DELIVERABLE

Done when: a selectivity number for R-32/R-125 in at least one framework, traceable to pure-component fits you understand.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ Whiteboard why an azeotrope blocks distillation, and how IAST gets mixture behavior from pure-component isotherms.

Read: Myers & Prausnitz (1965), the original IAST paper; one recent MOF fluorocarbon-separation study.

1.2 GCMC ground truth & its statistical mechanics

~1 week

WHAT YOU'LL WALK AWAY ABLE TO DO

- Run a GCMC screen for both refrigerants
- Explain every term in the GCMC acceptance rule

THE IDEA

Grand-canonical Monte Carlo fixes chemical potential, volume and temperature and lets the number of guest molecules fluctuate — exactly the physical situation of a framework in equilibrium with a gas reservoir. The simulation proposes insertions, deletions and moves, accepting each with a Metropolis probability that encodes the chemical potential (via fugacity). Understanding this is what separates “I ran RASPA” from “I know what RASPA computed.”

BUILD IT

- Screen ~500–2000 CoRE MOF 2019 structures for R-32 and R-125 uptake.
- For one structure, write out the insertion-acceptance probability and identify the fugacity term.
- Record uptake at the conditions feeding your IAST selectivity from 1.1.

DELIVERABLE

Done when: single-component uptakes for both gases across your screening subset.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ Write the GCMC insertion acceptance probability and explain why you specify fugacity rather than pressure at your chosen state point.

1.3 Refrigerant force fields vs. your MLIP

~4–5 days

WHAT YOU'LL WALK AWAY ABLE TO DO

- Justify the force field behind every label with an MLIP comparison

THE IDEA

Your GCMC labels are only as trustworthy as the guest–framework interactions. You already validate force fields against UMA/fairchem along binding paths for CF_4/CH_4 ; here you do the same for R-32 and R-125 so

the dataset rests on a defensible energy model rather than an inherited parameter set.

BUILD IT

- Compute MLIP reference energies along a few binding/approach paths for each refrigerant.
- Overlay the classical force field; quantify and document deviations.
- Pick the force field you will commit to and write the one-paragraph justification.

DELIVERABLE

Done when: a short FF-validation note that becomes the data-quality paragraph of Paper 1.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ In two sentences, defend your chosen force field using the MLIP comparison.

1.4 Featurize & curate a leakage-aware dataset

~1 week

WHAT YOU'LL WALK AWAY ABLE TO DO

- Produce a clean (features, selectivity) table
- Demonstrate and prevent train/test leakage by structure family

THE IDEA

Models learn from **descriptors**: geometric ones (pore-limiting diameter, largest cavity diameter, void fraction, surface area from Zeo++) and chemical ones (revised autocorrelations, RACs, from molSimplify). The subtle trap is **leakage**: if near-identical frameworks land in both train and test, your error estimate is optimistic and your screen will disappoint in the wild. Grouped splitting by structure family is the honest fix.

BUILD IT

- Run Zeo++ and RACs; assemble a tidy labeled table with provenance columns.
- Build two splits: naive random, and family-grouped.
- Train any quick baseline on both and show the random split inflates the score.

DELIVERABLE

Done when: a versioned dataset plus a figure showing the leakage gap between split strategies.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ Quantify the optimism of the random split vs. the grouped split and explain, mechanistically, where the leak comes from.

Read: a CoRE MOF descriptor paper; the molSimplify RAC paper.

MODULE

2 · Surrogate model + uncertainty

~4 weeks

You build a fast, honest predictor of selectivity — first with trees, then a graph neural network — and wrap it in calibrated uncertainty. The UQ here is not a side quest: in Module 5 it becomes the acquisition signal that decides which generated candidates are worth an expensive GCMC check. This module is publishable on its own.

Reuses what you already have:

Your labeled dataset (Module 1); Python/PyTorch from Module 0.

2.1 Dimensionality reduction the honest way

~3 days

WHAT YOU'LL WALK AWAY ABLE TO DO

- Implement PCA from its definition and interpret components physically

THE IDEA

PCA finds orthogonal directions of maximum variance — the eigenvectors of the feature covariance matrix. Implementing it once from the eigendecomposition (rather than calling a black box) makes “latent space” in the VAE far less mysterious, because a VAE latent is a learned, nonlinear cousin of the same idea.

BUILD IT

- Implement PCA via eigendecomposition of the covariance matrix; verify against sklearn.
- Run UMAP on your descriptors and color points by selectivity to see structure.
- Name what the leading components mean for your materials.

DELIVERABLE

Done when: a 2-D embedding of your dataset with a physical reading of the top axes.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ Explain what the first two principal components represent for your descriptors, and why variance-maximizing directions are useful.

2.2 Baseline regression & screening metrics

~4 days

WHAT YOU'LL WALK AWAY ABLE TO DO

- Train a solid tree baseline and pick the right headline metric

THE IDEA

For screening you care about **ranking** — finding the best materials — more than absolute error. That makes Spearman rank correlation often the truer headline than MAE. Bias–variance and proper cross-validation tell you whether the model will hold up out of sample.

BUILD IT

- Train gradient-boosted trees with grouped cross-validation.
- Report MAE, R-squared, and Spearman; inspect the parity and residual plots.
- Decide and justify your primary metric for the downstream loop.

DELIVERABLE

Done when: a baseline with cross-validated metrics and a stated headline metric.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ Argue when Spearman beats MAE as the headline for a screen, and when it does not.

2.3 A graph neural network from message passing up

~1.5 weeks

WHAT YOU'LL WALK AWAY ABLE TO DO

- Hand-build one message-passing layer and confirm permutation invariance
- Train a CGCNN-style model and beat (or understand why not) the baseline

THE IDEA

Crystals and molecules are graphs: atoms are nodes, bonds/contacts are edges. A **message-passing** layer updates each node from a permutation-invariant aggregate of its neighbors. Invariance matters because relabeling atoms must not change a prediction. Building one layer by hand demystifies every graph model you will touch afterward.

BUILD IT

- On a 5-node toy graph, implement gather → aggregate (sum) → update by hand; permute node order and confirm outputs are unchanged.
- Move to PyTorch Geometric and train a CGCNN-style network on your structures.
- Compare against the tree baseline; explain any gap from descriptors vs. learned features.

One layer, conceptually — implement, don't import, the first time

```

m_ij = phi(h_i, h_j, e_ij)      # message on each edge
h_i' = gamma(h_i, SUM_j m_ij)   # permutation-invariant update
# sum/mean are invariant; concatenation of neighbors is NOT

```

DELIVERABLE

Done when: a trained GNN with a numerical permutation-invariance test in your tests/ folder.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ From a blank file, write one message-passing layer and explain why sum aggregation is permutation-invariant but neighbor concatenation is not.

Read: CGCNN (Xie & Grossman, 2018).

2.4 Uncertainty quantification & calibration

~1 week

WHAT YOU'LL WALK AWAY ABLE TO DO

- Produce calibrated predictive uncertainty with guaranteed coverage
- Distinguish epistemic from aleatoric uncertainty in your own data

THE IDEA

Aleatoric uncertainty is irreducible noise in the data; **epistemic** uncertainty is the model's ignorance, which shrinks with more data — and it is epistemic uncertainty that drives active learning in Module 5. A model is **calibrated** if its stated confidence matches reality; a **reliability diagram** and the expected calibration error measure that. **Split conformal prediction** turns any model into one with a finite-sample coverage guarantee in about fifteen lines.

BUILD IT

- Train a deep ensemble (several seeds); use mean and spread as prediction and uncertainty.
- Implement a reliability diagram and expected calibration error by hand.
- Implement split conformal prediction from scratch and verify empirical coverage.

Split conformal — the whole idea

```

# calibrate on held-out residuals; q = (1-alpha) quantile of |y - yhat|
qhat = quantile(abs(y_cal - pred(x_cal)), 1-alpha)
interval(x) = [pred(x) - qhat, pred(x) + qhat] # ~ (1-alpha) coverage

```

DELIVERABLE

Done when: a calibrated surrogate with a reliability diagram on the grouped test set.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ Implement split conformal and a reliability diagram blank-file; explain epistemic vs. aleatoric with a concrete example from your dataset.

Read: Lakshminarayanan et al. (deep ensembles); a short conformal-prediction tutorial.

MODULE

3 • Generative models

5–6 weeks

The headline skill and the biggest gap to close. You learn latent-variable and diffusion models by building each on a tiny toy where the math is visible, then apply them to designing linkers, finally conditioning generation on target selectivity. This is the capability the DeepMind and generative-design postings lead with.

Reuses what you already have:

Your KL/ELBO groundwork (0.2), training loop (0.3), and surrogate scorer (Module 2).

3.1 Representing a linker

~4 days

WHAT YOU'LL WALK AWAY ABLE TO DO

- Encode linkers as SMILES and as graphs, with a validity check

THE IDEA

Generation needs a representation. **SMILES** strings are compact but most random strings are invalid molecules; **molecular graphs** encode validity more naturally but are higher-dimensional. Quantifying how rare valid random outputs are motivates why generative models must *learn* the manifold of real molecules.

BUILD IT

- Build a SMILES tokenizer and a graph featurizer for your linker library.
- Write an `is_valid()` check (parse + sanitize).
- Measure the fraction of random token sequences that are valid molecules.

DELIVERABLE

Done when: two interchangeable linker representations and a validity oracle.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ Report the random-validity rate and explain why it makes naive sampling hopeless.

3.2 The VAE from the ELBO up

~1.5 weeks

WHAT YOU'LL WALK AWAY ABLE TO DO

- Derive the ELBO and implement the reparameterization trick
- Sample novel linkers and quantify validity/novelty
- Recognize and mitigate posterior collapse

THE IDEA

A **variational autoencoder** learns a smooth latent space by maximizing the evidence lower bound (ELBO) = reconstruction quality minus KL(approximate posterior || prior). That KL is exactly the quantity you coded in 0.2. The **reparameterization trick** rewrites a random sample as a deterministic function of the parameters plus fixed noise, so gradients can flow through sampling. **Posterior collapse** — the latent being ignored — is the classic failure to watch for.

BUILD IT

- Derive the ELBO on paper, naming the reconstruction and KL terms.
- Implement a VAE on 2-D toy data; visualize the latent space and interpolations.
- Move to a small VAE over linker SMILES/graphs; sample and measure validity and novelty.

Reparameterization — the one line that makes it trainable

```
mu, logvar = encoder(x)
z = mu + exp(0.5*logvar) * randn_like(mu)    # gradient flows through mu, logvar
loss = recon(decoder(z), x) + beta * kl(mu, logvar)  # reuse your 0.2 KL
```

DELIVERABLE

Done when: a VAE that emits mostly-valid novel linkers from a sampled latent.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ Derive the ELBO and explain precisely why the reparameterization trick is needed to backpropagate through a sampling step.

3.3 Diffusion from scratch

~2 weeks

WHAT YOU'LL WALK AWAY ABLE TO DO

- Implement the forward and reverse diffusion processes on a 2-D toy
- State the noise-prediction objective and why it works
- Read CDVAE/MOFDiff as your architecture references

THE IDEA

A **diffusion model** gradually adds Gaussian noise to data (the fixed forward process) and learns to reverse it. Training reduces to a startlingly simple objective: predict the noise that was added at a random timestep. Predicting that noise is equivalent to learning the score (gradient of log-density), which is why reverse

sampling walks noise back toward the data manifold. The 2-D point-cloud toy lets you literally watch structure emerge from noise.

BUILD IT

- On a 2-D mixture of Gaussians, implement the forward noising in closed form, train a noise predictor, and sample by running the reverse process; animate it.
- Study the variance schedule and how it shapes sample quality.
- Scale toward 3-D building-block generation; read CDVAE and MOFDiff for how building blocks are diffused and then assembled into frameworks.

The DDPM core you will implement

```
x_t = sqrt(abar_t)*x0 + sqrt(1-abar_t)*eps      # forward, eps ~ N(0,I)
loss = mse(model(x_t, t), eps)                 # predict the noise
# sampling: iterate the learned reverse step from t=T down to 0
```

DELIVERABLE

Done when: a working 2-D DDPM you fully understand, plus a plan for 3-D building blocks.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ Implement DDPM forward and reverse on a 2-D mixture from a blank file; write the training objective and explain why predicting noise approximates the score.

Read: DDPM (Ho et al., 2020); CDVAE (Xie et al., 2022); MOFDiff.

3.4 Property-conditioned generation

~1 week

WHAT YOU'LL WALK AWAY ABLE TO DO

- Condition a generator on target selectivity and prove the shift

THE IDEA

Unconditional generation samples the whole distribution; you want *high-selectivity* linkers. **Classifier-free guidance** trains the model both with and without the condition and, at sampling time, extrapolates toward the conditioned direction — a simple, strong way to steer generation toward a property target.

BUILD IT

- Add a selectivity condition to your generator (VAE or diffusion).
- Implement classifier-free guidance.
- Use your Module-2 surrogate to score conditional vs. unconditional batches.

DELIVERABLE

Done when: a property-conditioned generator with evidence it shifts the output distribution.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ Show, with a plot, that conditioning moves generated candidates toward higher predicted selectivity, and explain the guidance mechanism.

MODULE

4 · Physics-informed steering / RL

3–4 weeks

You make generation answer to physics. Reinforcement learning, built up from a bandit and REINFORCE, lets you fine-tune the generator against a reward built from your own MLIP — the exact “physics-informed reward from machine-learned potentials” bullet, and a near-free win because you already run that potential.

Reuses what you already have:

The log-derivative trick (0.2); your UMA/fairchem MLIP and Boltzmann-weighting experience.

4.1 RL fundamentals through code

~1 week

WHAT YOU'LL WALK AWAY ABLE TO DO

- Implement REINFORCE from scratch and reduce its variance with a baseline

THE IDEA

Reinforcement learning maximizes expected reward over actions drawn from a policy. The **policy-gradient** estimator is precisely the log-derivative trick from 0.2: the gradient is an expectation of (grad log policy) times reward. The estimator is noisy; a **baseline** subtracted from the reward cuts variance without adding bias.

BUILD IT

- Solve a multi-armed bandit to internalize exploration vs. exploitation.
- Implement REINFORCE on a tiny discrete environment from a blank file.
- Add a baseline and measure the variance reduction empirically.

REINFORCE — your 0.2 identity, applied

```
loss = -(logprob(action) * (reward - baseline)).mean()
loss.backward(); opt.step()    # grad log pi * advantage
```

DELIVERABLE

Done when: a from-scratch REINFORCE agent with a working baseline.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ Write the REINFORCE update and explain why a baseline lowers variance but not bias.

4.2 Reward design with your MLIP

~1 week

WHAT YOU'LL WALK AWAY ABLE TO DO

- Turn MLIP energetics into a defensible scalar reward

THE IDEA

A generator can produce high-selectivity molecules that are physically nonsense. The fix is a reward that prices in stability: relaxation energy, strain, and a stability proxy from UMA, combined with predicted selectivity and validity. Your existing Boltzmann-weighting intuition transfers directly to normalizing and combining these terms.

BUILD IT

- Define reward = $f(\text{MLIP stability, predicted selectivity, validity})$; document trade-offs.
- Normalize terms onto comparable scales; decide weighting.
- Stress-test the reward on known-good and known-bad structures.

DELIVERABLE

Done when: a documented reward function with each term justified.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ Defend every term in your reward and name one concrete way it could be gamed.

4.3 Steering the generator

~1 week

WHAT YOU'LL WALK AWAY ABLE TO DO

- Bias generation toward stable, selective candidates and catch reward hacking

THE IDEA

Start cheap: reward-guided / best-of-N sampling (generate many, keep the best by reward). Then graduate to REINFORCE fine-tuning of the generator. **Reward hacking** — the model exploiting flaws in the reward rather than solving the task — is the central risk, and catching it is itself a demonstrable research skill.

BUILD IT

- Implement best-of-N selection over generated batches.
- Fine-tune the generator with REINFORCE against the MLIP-informed reward.
- Find and document a reward-hacking case (high reward, MLIP-unstable) and your catch.

DELIVERABLE

Done when: a steered generator plus a written reward-hacking post-mortem.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ Exhibit a reward-hacking example you caught and explain the guardrail you added.

Read: the classifier-free-guidance / RLHF connection; any treatment of reward hacking in design.

MODULE

5 · Closed-loop validation & evaluation

3–4 weeks

You close the loop: generated candidates are filtered by the MLIP, ranked by the surrogate, validated by GCMC, and fed back to retrain — with your simulations standing in for an automated lab. You also build the evaluation report card every generative-design posting names as a deliverable.

Reuses what you already have:

Surrogate + UQ (Module 2), generator (Module 3), reward (Module 4), GCMC and free energy (your toolkit).

5.1 Evaluation metrics from scratch

~1 week

WHAT YOU'LL WALK AWAY ABLE TO DO

- Build a report card of validity, novelty, uniqueness, stability, synthesizability

THE IDEA

Each metric catches a distinct failure: **validity** (is it a real molecule), **uniqueness** (not the same thing repeatedly), **novelty** (distance from the training set), **stability** (survives an MLIP relaxation; a few get full Frenkel–Ladd free energies — your method), and **synthesizability** (an SA score plus building-block availability). A model can ace the first two and fail the last, which is why the full card matters.

BUILD IT

- Implement validity, uniqueness and novelty by hand on your generated set.
- Add stability via UMA relaxation and synthesizability via an SA proxy.
- Assemble a single report-card function that scores any batch.

DELIVERABLE

Done when: a one-call evaluation harness producing the full five-metric card.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ Explain how a generator can score high on validity and novelty yet fail synthesizability, and how your card exposes it.

5.2 Active learning loop

~1 week

WHAT YOU'LL WALK AWAY ABLE TO DO

- Use surrogate uncertainty to choose which candidates get expensive validation

THE IDEA

Validation by GCMC is costly, so spend it wisely. **Acquisition functions** balance exploring uncertain regions against exploiting promising ones; the epistemic uncertainty you built in 2.4 is the natural exploration signal. This is the same machinery as Bayesian optimization, scoped to your loop.

BUILD IT

- On a synthetic regression, show uncertainty sampling beats random at equal budget.
- Wire surrogate uncertainty into a candidate-selection step.
- Define the retrain-after-acquisition cycle.

DELIVERABLE

Done when: an acquisition step that measurably outperforms random selection.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ Explain how epistemic uncertainty drives acquisition, and implement uncertainty sampling from a blank file.

5.3 Wire the full loop

~1.5 weeks

WHAT YOU'LL WALK AWAY ABLE TO DO

- Run one complete generate-validate-retrain iteration end to end

THE IDEA

Everything now connects: the generator proposes, the MLIP filters for stability, the surrogate ranks with uncertainty, GCMC validates the top-k, and those true labels retrain the surrogate. One honest iteration that improves the true selectivity of the top candidates is the whole thesis of Paper 2 in miniature.

BUILD IT

- Compose: generate → MLIP filter → surrogate rank (with UQ) → GCMC validate top-k → retrain.
- Log the true (GCMC) selectivity of the top candidates before and after.
- Capture every artifact so the iteration is reproducible.

The loop, in one breath

```
cands = generate(cond=high_selectivity)
cands = [c for c in cands if mlip_stable(c)]
ranked = surrogate.rank(cands, with_uncertainty=True)
truth = gcmc(ranked[:k])
surrogate.update(truth)           # active-learning retrain
```

DELIVERABLE

Done when: a measured improvement in GCMC selectivity of top candidates after one loop.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ Run one full iteration and show the before/after true-selectivity gain; explain which component contributed most.

MODULE

6 · Orchestration, agents & portfolio

2–3 weeks

You make the loop reproducible, optionally hand the steering wheel to an LLM agent, and package everything as a portfolio that reads one-to-one against the job descriptions — with your closed-book reimplementations committed as visible proof of depth.

Reuses what you already have:

The full pipeline (Modules 1–5); your HPC experience.

6.1 Reproducible pipeline

~1 week

WHAT YOU'LL WALK AWAY ABLE TO DO

- Wrap the loop as a config-driven, containerized, HPC-deployable workflow

THE IDEA

A workflow engine (Snakemake or Prefect) turns your loop into named, cached, re-runnable stages with explicit dependencies — the “modular, reproducible pipelines deployable on HPC and cloud” bullet, made literal.

BUILD IT

- Express each loop stage as a workflow rule with declared inputs/outputs.
- Containerize the environment; parameterize entirely through config.
- Document a one-command run.

DELIVERABLE

Done when: a collaborator runs one loop iteration from your README without you present.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ Hand the repo to someone and confirm they reproduce an iteration unaided.

6.2 Optional agentic layer

~4–5 days

WHAT YOU'LL WALK AWAY ABLE TO DO

- Let a thin LLM agent propose the next batch, read results, and decide

THE IDEA

A lightweight orchestrator (you can use the Claude API) reads the last iteration's metrics, proposes the next conditioned generation, and triggers validation — covering the “closed-loop autonomous / multi-agent” bullets without adding any physics. Keep the human as approver.

BUILD IT

- Expose your loop stages as callable tools.
- Write an agent prompt that plans the next batch from results.
- Run one agent-driven iteration with you only approving.

DELIVERABLE

Done when: one iteration completed under agent control with human sign-off.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ Demonstrate the agent completing a loop where you approve but never code the step.

6.3 Portfolio & job-description mapping

~4 days

WHAT YOU'LL WALK AWAY ABLE TO DO

- Package the work so each module visibly answers a posting

THE IDEA

The same repository is both a methods contribution and a hiring artifact. Make the map explicit, and commit your from-scratch reimplementations — they are the evidence that you can do this when the AI is closed.

BUILD IT

- Write a clean README and a short per-module writeup.
- Commit the closed-book toy implementations as a visible “foundations” folder.
- Add the skill-to-JD mapping table (next page) to the README.

DELIVERABLE

Done when: a portfolio repo a hiring manager can skim and map to their own posting.

CLOSED-BOOK CHECKPOINT (AI + notes shut)

- ✓ For three target postings, point to the exact module/artifact that satisfies each key bullet.

Skill → job-description mapping

Every module exists because a posting asks for it. Keep this table in your README.

You build (module)	JD bullet it satisfies
Reproducible pipeline (0, 6)	modular, reproducible pipelines on HPC / cloud
Dataset curation + IAST/GCMC (1)	curate, preprocess, manage large-scale datasets
Surrogate + GNN (2)	ML on materials data; atomic descriptors into ML workflows
UQ + calibration + conformal (2.4)	uncertainty quantification; calibrated probabilistic prediction
VAE + diffusion + conditioning (3)	generative models for structure generation, property-conditioned design, CSP
MLIP reward + REINFORCE (4)	RL / alignment with physics-informed reward from MLIPs
Evaluation report card (5.1)	validity, novelty, uniqueness, stability, synthesizability frameworks
Closed loop + active learning (5)	coupling to observables for closed-loop verification
Agentic orchestrator (6.2)	embed into LLM multi-agent systems for closed-loop autonomy

Reuse ledger — what you already have

GCMC / RASPA · MLIPs (UMA / fairchem) · Frenkel–Ladd free energy · force-field fitting · Boltzmann weighting · MSD / transport · Packmol / Moltemplate / LAMMPS / PLUMED · HPC · Python / Bash. Every physics component above is something you already do; the course only adds the ML layer on top.

The one honest test

After AI helps you build any piece, could you rebuild its core from a blank file and whiteboard it to someone? Yes means it accelerated you. No means it replaced your learning and you now have a silent gap. Build toward yes, module by module.